.NET TECHNOLOGIES (01CE1523)
Lab Manual

Name: AYUSH SHARMA
Enrolment No: 92400103460
Class: 5EC1
Batch: A

Experiment 8

AIM: Develop Inventory Management Module using ADO.NET & CRUD Operations.

Source: Database/Setup.sql

```
USE master;
GO
IF DB_ID(N'InventoryPractical8') IS NULL
    CREATE DATABASE InventoryPractical8;
GO
USE InventoryPractical8;
GO
IF OBJECT_ID(N'dbo.Products', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Products (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        Name NVARCHAR(80) NOT NULL,
        Category NVARCHAR(50) NOT NULL,
        Quantity INT NOT NULL CHECK (Quantity BETWEEN 0 AND 1000000),
        Price DECIMAL(18,2) NOT NULL CHECK (Price BETWEEN 0.01 AND 9999999.99)
    );
END;
GO
```

Source: Models/Product.cs

```
using System.ComponentModel.DataAnnotations;

namespace InventoryManagement.Models
{
    public class Product
    {
        public int Id { get; set; }
        [Required, StringLength(80)]
        [Display(Name = "Product Name")]
        public string Name { get; set; }
        [Required, StringLength(50)]
        public string Category { get; set; }
        [Range(0, 1000000)]
        public int Quantity { get; set; }
        [Range(typeof(decimal), "0.01", "9999999.99")]
        public decimal Price { get; set; }
    }
}
```

Source: Data/ProductRepository.cs

```
using System.Collections.Generic;
using System.Configuration;
using System.Data;
using System.Data.SqlClient;
using InventoryManagement.Models;

namespace InventoryManagement.Data
{
    public class ProductRepository
    {
        private readonly string connectionString =
            ConfigurationManager.ConnectionStrings["InventoryDb"].ConnectionString;

        public List<Product> GetAll()
        {
            var products = new List<Product>();
            using (var connection = new SqlConnection(connectionString))
            using (var command = new SqlCommand(
                "SELECT Id, Name, Category, Quantity, Price FROM Products ORDER BY
Id", connection))
            {
                connection.Open();
                using (var reader = command.ExecuteReader())
                    while (reader.Read())
                        products.Add(ReadProduct(reader));
            }
            return products;
        }

        public Product GetById(int id)
        {
            using (var connection = new SqlConnection(connectionString))
            using (var command = new SqlCommand(
                "SELECT Id, Name, Category, Quantity, Price FROM Products WHERE
Id=@Id", connection))
            {
                command.Parameters.Add("@Id", SqlDbType.Int).Value = id;
                connection.Open();
                using (var reader = command.ExecuteReader())
                    return reader.Read() ? ReadProduct(reader) : null;
            }
        }

        public void Add(Product product)
        {
            Execute("INSERT INTO Products (Name,Category,Quantity,Price) " +
```

Source: Data/ProductRepository.cs (continued)

```
                "VALUES (@Name,@Category,@Quantity,@Price)", product);
        }

        public bool Update(Product product)
        {
            return Execute("UPDATE Products SET Name=@Name,Category=@Category," +
                "Quantity=@Quantity,Price=@Price WHERE Id=@Id", product) > 0;
        }

        private int Execute(string sql, Product product)
        {
            using (var connection = new SqlConnection(connectionString))
            using (var command = new SqlCommand(sql, connection))
            {
                command.Parameters.Add("@Id", SqlDbType.Int).Value = product.Id;
                command.Parameters.Add("@Name", SqlDbType.NVarChar, 80).Value =
product.Name;
                command.Parameters.Add("@Category", SqlDbType.NVarChar, 50).Value =
product.Category;
                command.Parameters.Add("@Quantity", SqlDbType.Int).Value =
product.Quantity;
                var price = command.Parameters.Add("@Price", SqlDbType.Decimal);
                price.Precision = 18;
                price.Scale = 2;
                price.Value = product.Price;
                connection.Open();
                return command.ExecuteNonQuery();
            }
        }

        public bool Delete(int id)
        {
            using (var connection = new SqlConnection(connectionString))
            using (var command = new SqlCommand("DELETE FROM Products WHERE Id=@Id",
 connection))
            {
                command.Parameters.Add("@Id", SqlDbType.Int).Value = id;
                connection.Open();
                return command.ExecuteNonQuery() > 0;
            }
        }

        private static Product ReadProduct(SqlDataReader reader)
        {
            return new Product {
                Id = reader.GetInt32(0), Name = reader.GetString(1),
```

Source: Data/ProductRepository.cs (continued)

```
                Category = reader.GetString(2), Quantity = reader.GetInt32(3),
                Price = reader.GetDecimal(4)
            };
        }
    }
}
```

Source: Controllers/ProductController.cs

```
using System.Web.Mvc;
using InventoryManagement.Data;
using InventoryManagement.Models;

namespace InventoryManagement.Controllers
{
    public class ProductController : Controller
    {
        private readonly ProductRepository repository = new ProductRepository();
        public ActionResult Index() { return View(repository.GetAll()); }
        public ActionResult Create() { return View("Form", new Product()); }

        [HttpPost, ValidateAntiForgeryToken]
        public ActionResult Create([Bind(Include="Name,Category,Quantity,Price")]
Product product)
        {
            if (!ModelState.IsValid) return View("Form", product);
            repository.Add(product);
            TempData["Success"] = "Product added successfully.";
            return RedirectToAction("Index");
        }

        public ActionResult Edit(int id)
        {
            var product = repository.GetById(id);
            if (product == null) return HttpNotFound();
            return View("Form", product);
        }

        [HttpPost, ValidateAntiForgeryToken]
        public ActionResult Edit(int id,
            [Bind(Include="Name,Category,Quantity,Price")] Product product)
        {
            product.Id = id;
            if (!ModelState.IsValid) return View("Form", product);
            if (!repository.Update(product)) return HttpNotFound();
            TempData["Success"] = "Product updated successfully.";
```

Source: Controllers/ProductController.cs (continued)

```
            return RedirectToAction("Index");
        }

        public ActionResult Delete(int id)
        {
            var product = repository.GetById(id);
            if (product == null) return HttpNotFound();
            return View(product);
        }

        [HttpPost, ActionName("Delete"), ValidateAntiForgeryToken]
        public ActionResult DeleteConfirmed(int id)
        {
            if (!repository.Delete(id)) return HttpNotFound();
            TempData["Success"] = "Product deleted successfully.";
            return RedirectToAction("Index");
        }
    }
}
```

Source: Views/Product/Form.cshtml

```
@model InventoryManagement.Models.Product
@{
    var action = Model.Id == 0 ? "Create" : "Edit";
    ViewBag.Title = Model.Id == 0 ? "Add Product" : "Edit Product";
}
<h1>@ViewBag.Title</h1>
<p>Enter product details. Price is in INR.</p>
@using (Html.BeginForm(action, "Product", new { id = Model.Id },
    FormMethod.Post, new { @class = "panel" }))
{
    @Html.AntiForgeryToken()
    @Html.ValidationSummary(false, "Please correct the following errors:")
    <div class="field">@Html.LabelFor(m => m.Name)
        @Html.TextBoxFor(m => m.Name, new { maxlength = 80 })
        @Html.ValidationMessageFor(m => m.Name)</div>
    <div class="field">@Html.LabelFor(m => m.Category)
        @Html.TextBoxFor(m => m.Category, new { maxlength = 50 })
        @Html.ValidationMessageFor(m => m.Category)</div>
    <div class="field">@Html.LabelFor(m => m.Quantity)
        @Html.TextBoxFor(m => m.Quantity)
        @Html.ValidationMessageFor(m => m.Quantity)</div>
    <div class="field">@Html.LabelFor(m => m.Price)
        @Html.TextBoxFor(m => m.Price, "{0:0.00}")
        @Html.ValidationMessageFor(m => m.Price)</div>
```

Source: Views/Product/Form.cshtml (continued)

```
    <button type="submit">Save Product</button>
    @Html.ActionLink("Cancel", "Index", null, new { @class = "secondary" })
}
```

Source: Views/Product/Index.cshtml

```
@model List<InventoryManagement.Models.Product>
@{ ViewBag.Title = "Inventory"; }
<div class="heading"><div><h1>Inventory Management</h1>
<p>Manage product stock and prices.</p></div>
@Html.ActionLink("+ Add Product", "Create", null, new { @class = "button" })</div>
@if (TempData["Success"] != null)
{ <div class="success">@TempData["Success"]</div> }
<p><strong>Total Products: @Model.Count</strong></p>
<div class="panel table-wrap">
<table><thead><tr><th>ID</th><th>Product Name</th><th>Category</th>
<th>Quantity</th><th>Price (INR)</th><th>Actions</th></tr></thead>
<tbody>
@foreach (var product in Model)
{
    <tr><td>@product.Id</td><td>@product.Name</td><td>@product.Category</td>
    <td>@product.Quantity</td><td>@product.Price.ToString("F2")</td>
    <td>@Html.ActionLink("Edit", "Edit", new { id = product.Id })
    | @Html.ActionLink("Delete", "Delete", new { id = product.Id })</td></tr>
}
</tbody></table>
@if (Model.Count == 0) { <p>No products found. Add your first product.</p> }
</div>
```

Source: Views/Product/Delete.cshtml

```
@model InventoryManagement.Models.Product
@{ ViewBag.Title = "Delete Product"; }
<h1>Delete Product</h1>
<div class="panel">
    <h2>@Model.Name</h2>
    <p>Category: @Model.Category | Quantity: @Model.Quantity</p>
    <p>Price: INR @Model.Price.ToString("F2")</p>
    <p>Are you sure you want to delete this product?</p>
    @using (Html.BeginForm("Delete", "Product", new { id = Model.Id },
FormMethod.Post))
    {
        @Html.AntiForgeryToken()
        <button type="submit" class="danger">Delete Product</button>
        @Html.ActionLink("Cancel", "Index", null, new { @class = "secondary" })
    }
```

Source: Views/Product/Delete.cshtml (continued)

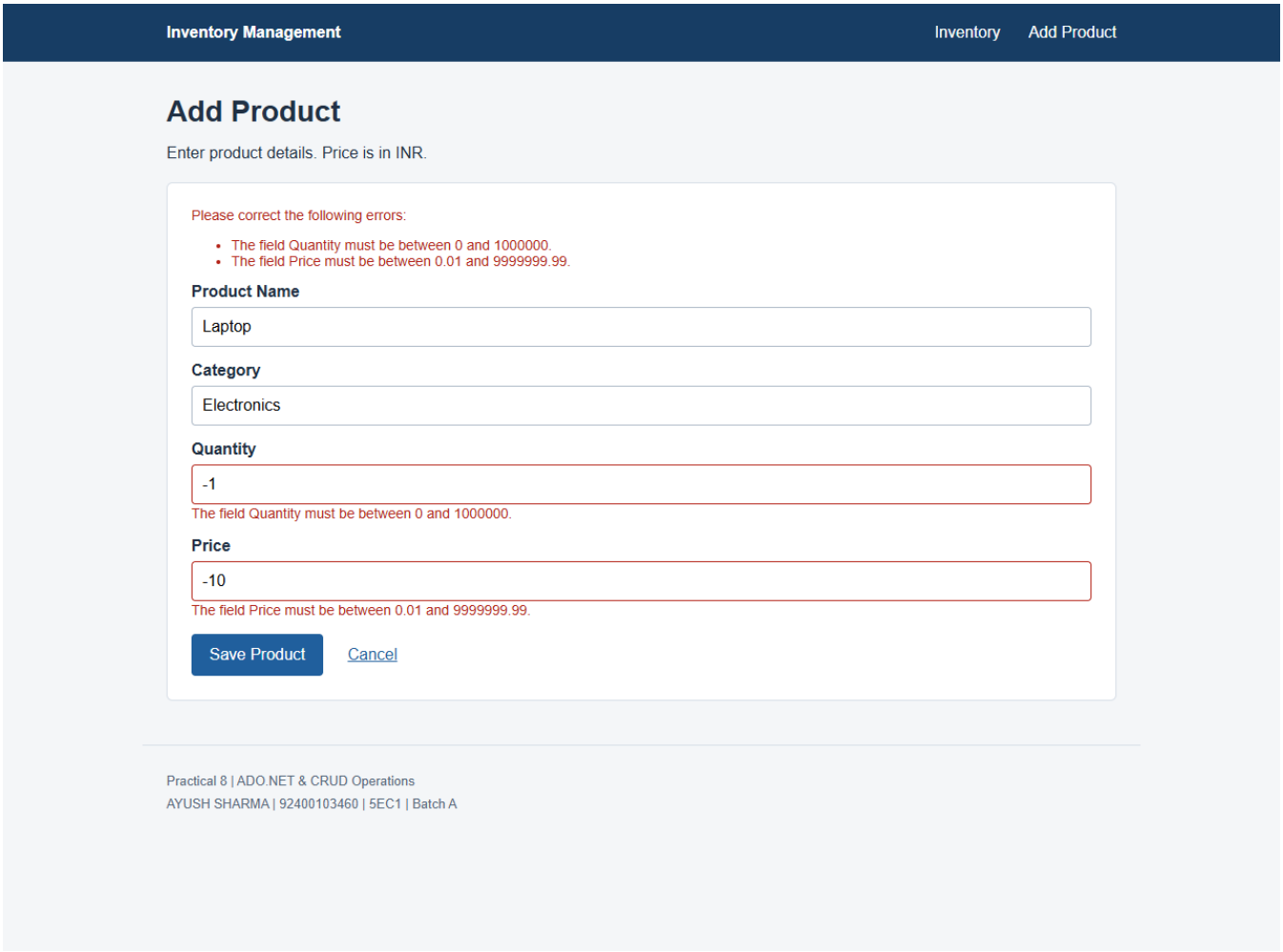
```
</div>
```

Source: Web.config

```
<?xml version="1.0"?>
<configuration>
  <connectionStrings>
    <add name="InventoryDb" connectionString="Data
Source=(LocalDB)\MSSQLLocalDB;Initial Catalog=InventoryPractical8;Integrated
Security=True;" providerName="System.Data.SqlClient" />
  </connectionStrings>
  <appSettings>
    <add key="webpages:Version" value="3.0.0.0" />
    <add key="webpages:Enabled" value="false" />
    <add key="ClientValidationEnabled" value="false" />
  </appSettings>
  <system.web>
    <compilation debug="true" targetFramework="4.8" />
    <httpRuntime targetFramework="4.8" />
  </system.web>
  <system.webServer>
    <modules runAllManagedModulesForAllRequests="true" />
    <validation validateIntegratedModeConfiguration="false" />
  </system.webServer>
  <runtime><assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
    <dependentAssembly><assemblyIdentity name="Microsoft.Web.Infrastructure"
publicKeyToken="31bf3856ad364e35" culture="neutral" /><bindingRedirect
oldVersion="0.0.0.0-2.0.0.0" newVersion="2.0.0.0" /></dependentAssembly>
  </assemblyBinding></runtime>
</configuration>
```

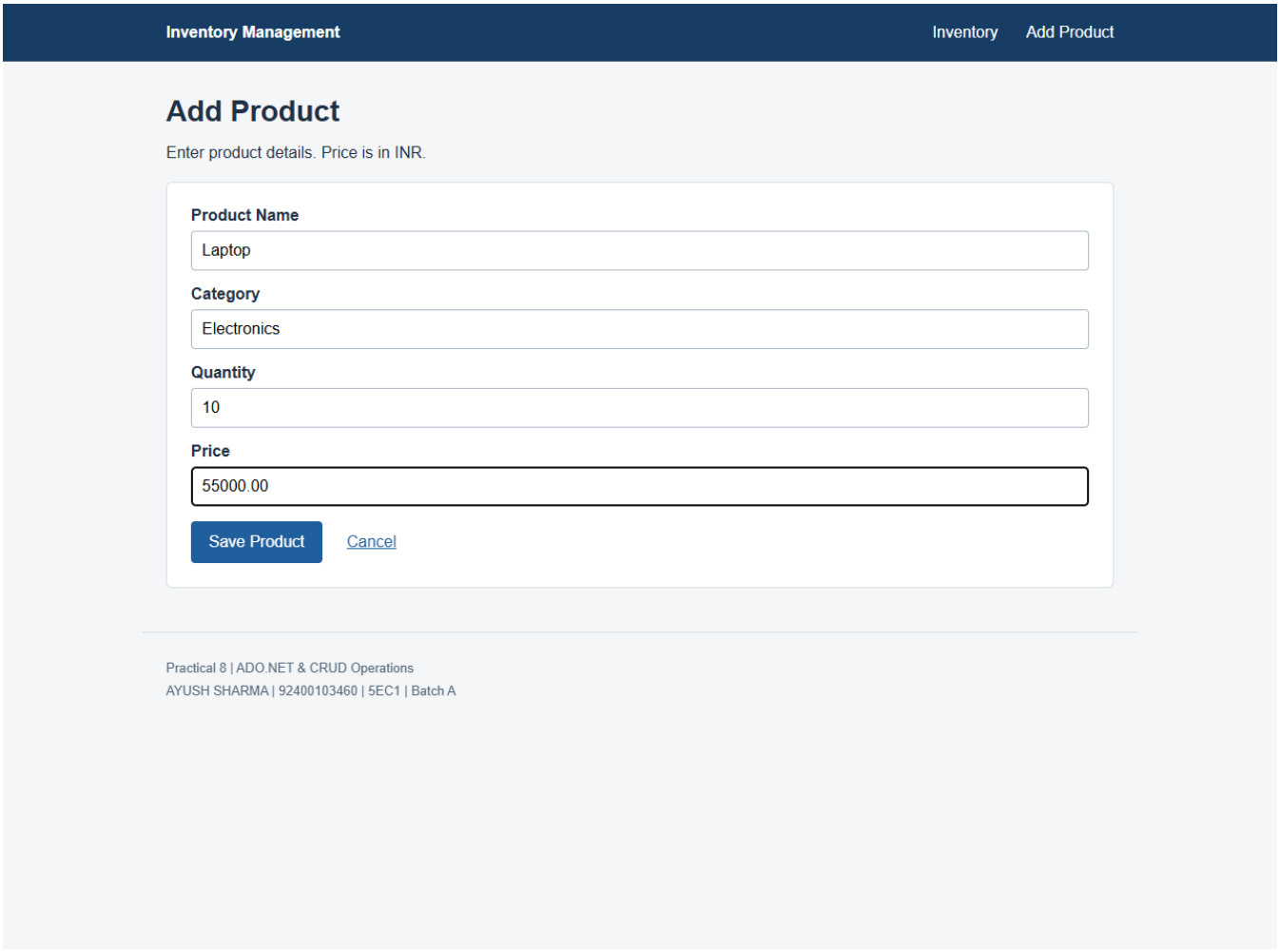
Output: Screenshot:

Input validation for stock quantity and product price



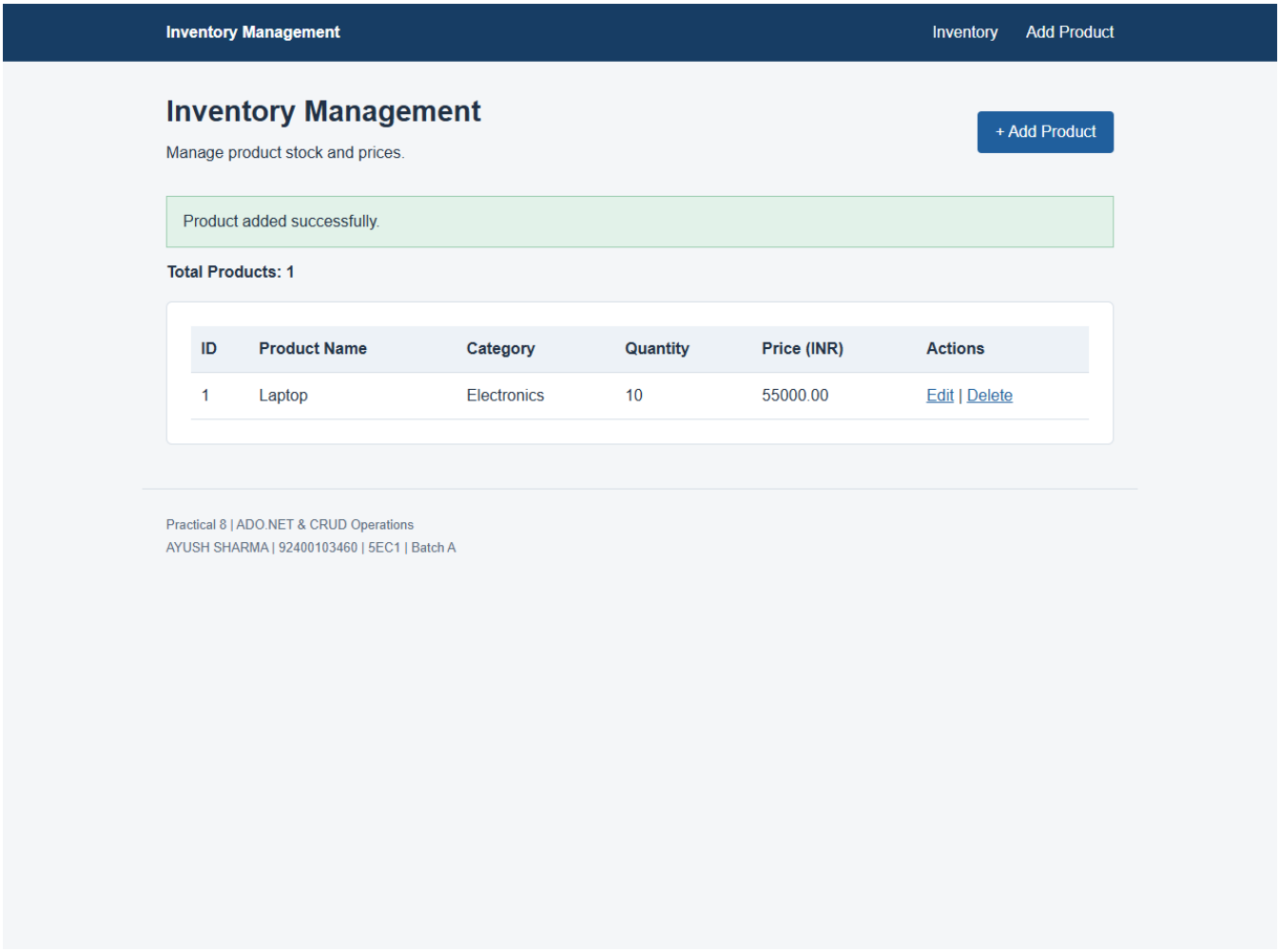
Output: Screenshot:

Create: Enter product details



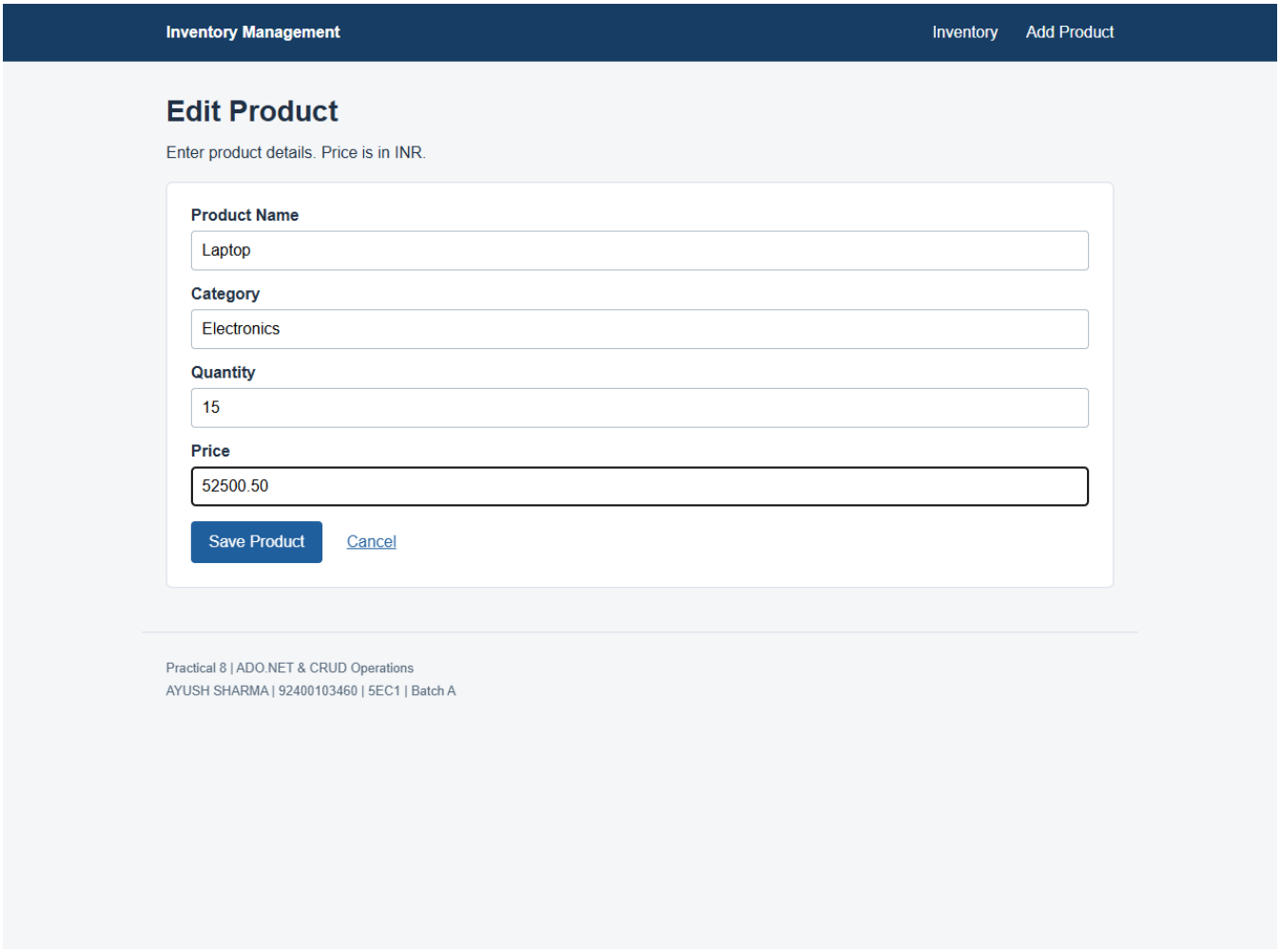
Output: Screenshot:

Read: Product saved in the SQL Server database



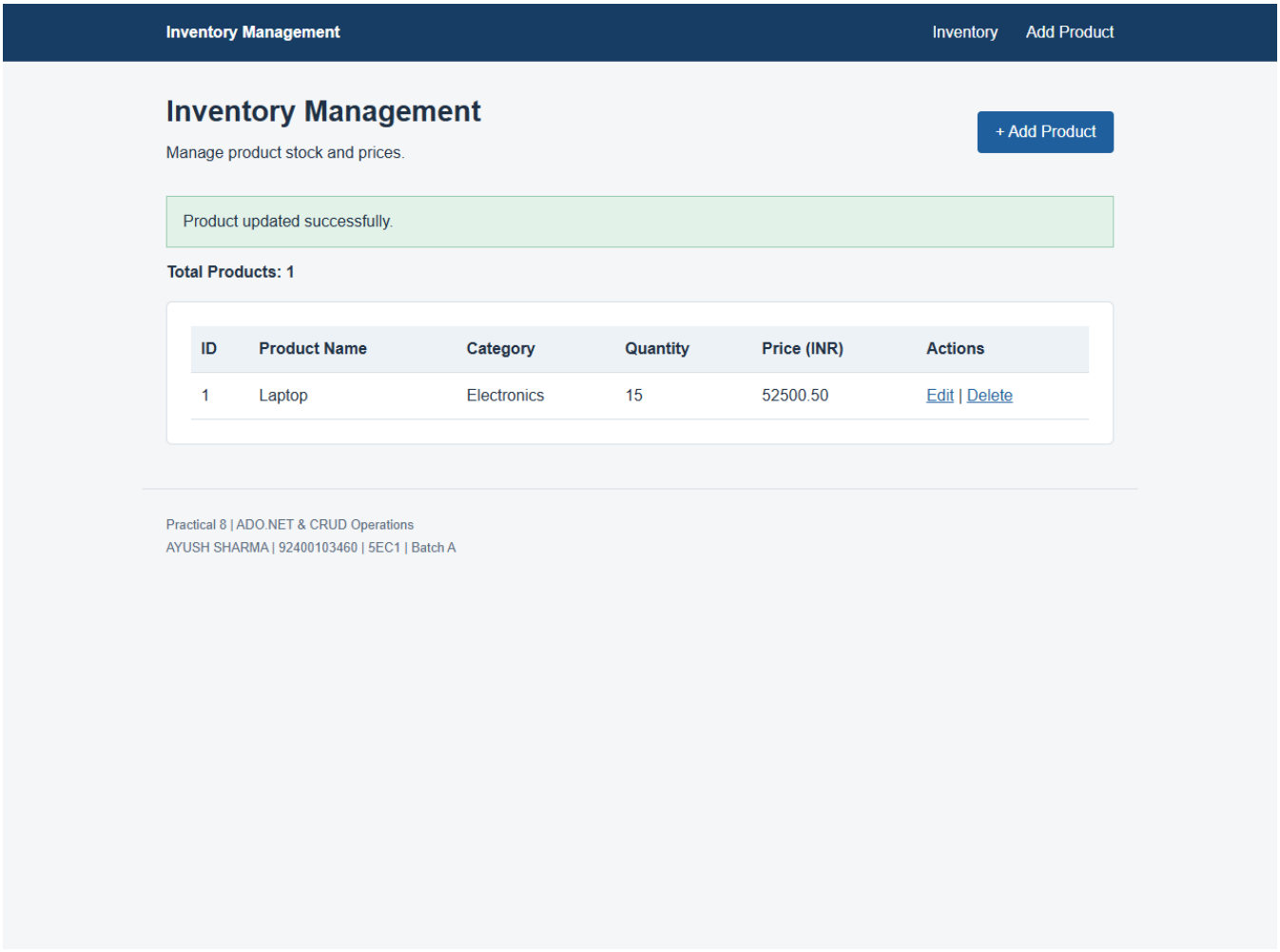
Output: Screenshot:

Update: Change stock quantity and price



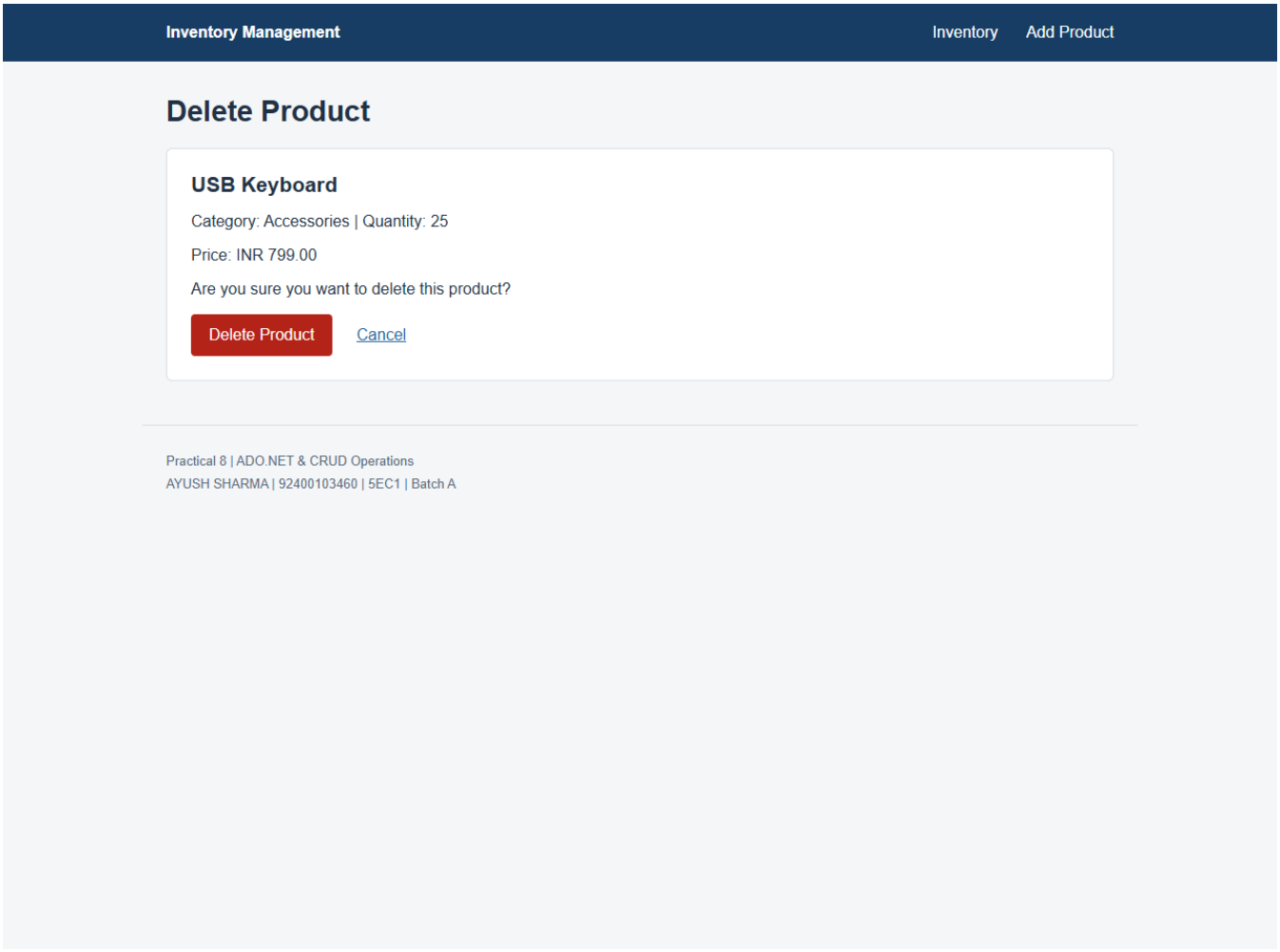
Output: Screenshot:

Read: Updated stock and price



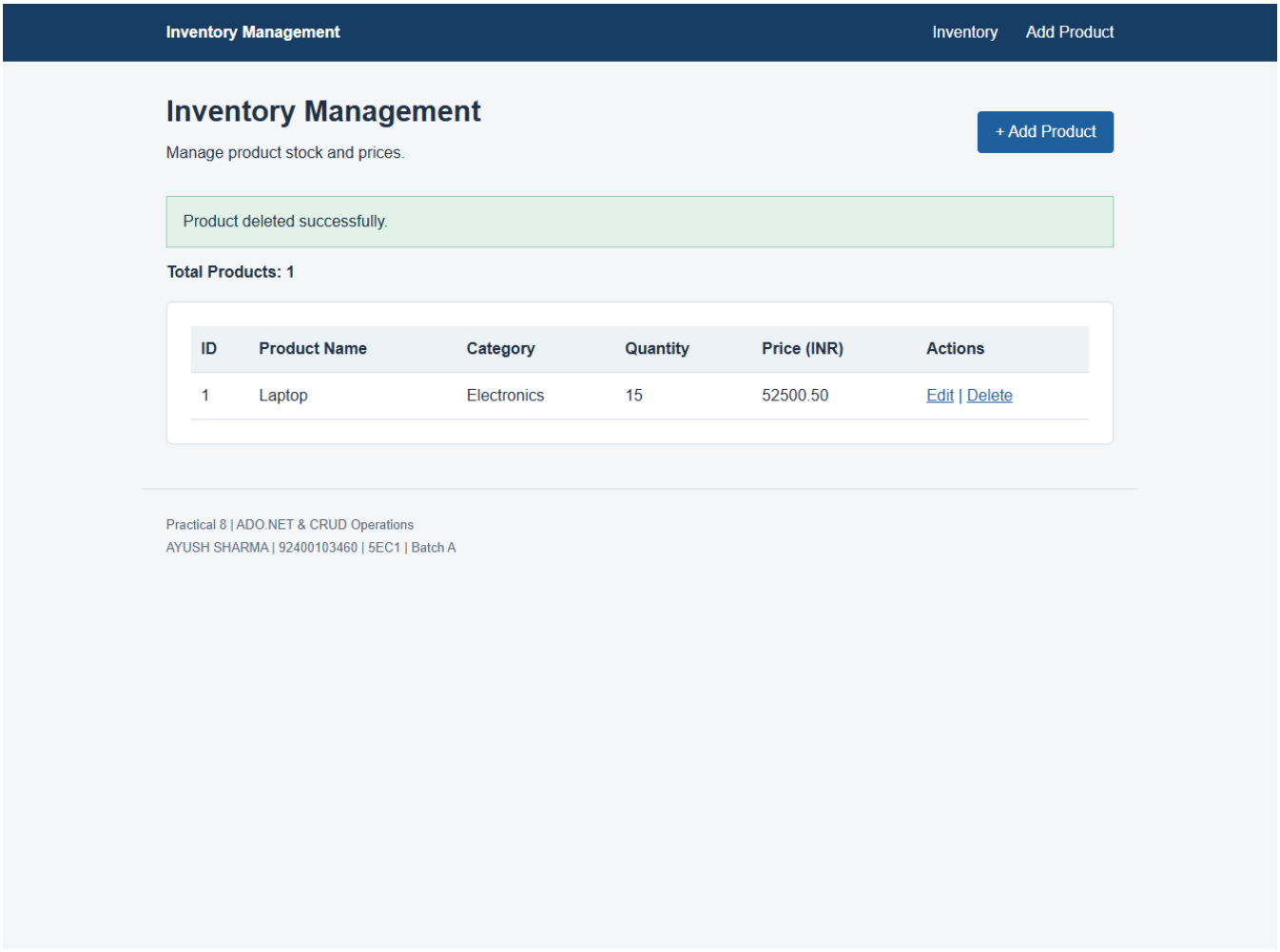
Output: Screenshot:

Delete: Confirm removal of a product



Output: Screenshot:

Read: Inventory after successful deletion



GitHub URL:

https://github.com/ius-sharma/Inventory-Management-Module